

# Lab 4 — What the monitoring saw

DDM501 · credit default risk API · team analysis

## 1. The signals and the question each one answers

Whether an applicant defaults is known a month later at the earliest, and for an applicant we declined it is never known. Accuracy is therefore not a production metric for this service. We watch four proxies instead. None of them proves the model is wrong; each is a reason to look.

| Signal       | Question it answers                                            | Metric                                                                       | Moves when                                                   |
|--------------|----------------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------|
| Input drift  | Do the applicants still look like the training population?     | <code>ml_feature_drift_psi</code> ,<br><code>ml_drift_score</code> (max PSI) | the inputs move, even if no decision changes                 |
| Output drift | Has the distribution of scores shifted?                        | <code>ml_prediction_score</code><br>(p50/p90/p99)                            | the inputs move in a direction the model cares about         |
| Decision mix | Is more work going to manual review, are more people declined? | <code>ml_decisions_total{decision}</code>                                    | scores cross the 0.30 / 0.60 business thresholds             |
| Fairness gap | Is one group flagged at a different rate from another?         | <code>ml_selection_rate{group}</code> ,<br><code>ml_fairness_gap</code>      | one group's inputs or treatment change relative to the other |

Service health (`http_requests_total`, `latency`, `up`) answers a fifth question, “is the service itself working?”, and lives on the Service Health dashboard. Every profile below returned 400/400 HTTP 200 with p95 latency under 70 ms: by service metrics, nothing ever went wrong.

## 2. Method

Stack: API, Prometheus (10 s scrape), Grafana and node-exporter from `docker-compose.yml`. Model: HistGradientBoosting, ROC AUC 0.7473 on the held-out 20%. Drift reference: 10 quantile bins per feature from the 24,000-row training split, outer edges open (`PAY_0` and `max_delay` collapse to 4 bins because they are discrete).

Each profile: restart the API (the window is in memory, so this empties it), send 400 requests with `scripts/load_test.py` (seed 501), then read `/monitoring`. Score mean and decision mix are counted by the load generator from the responses; PSI and the gap come from `/monitoring`. We ran every profile twice, once against the Docker stack and once against `uvicorn` on the host; the two runs agree to four decimals. The `normal`, `drifted 0.05`, `drifted 0.15` and `unfair` rows also match the course's reference table to four decimals. Our full-drift row is a little lower than the reference table (mean 0.590 vs 0.609); we did not find the cause and report our own number.

### 3. Results

| Profile                 | Score mean | REVIEW | DECLINE | Drift score | Status      | Fairness gap |
|-------------------------|------------|--------|---------|-------------|-------------|--------------|
| normal                  | 0.2355     | 14.2%  | 8.0%    | 0.0301      | stable      | 0.019        |
| drifted 0.05            | 0.2437     | 14.0%  | 8.5%    | 0.1885      | moderate    | 0.015        |
| drifted 0.15            | 0.2609     | 16.2%  | 9.8%    | 1.1518      | significant | 0.036        |
| drifted 1.0             | 0.5897     | 29.8%  | 54.8%   | 4.2510      | significant | 0.038        |
| unfair                  | 0.3966     | 15.8%  | 32.0%   | 0.3526      | significant | <b>0.722</b> |
| defaulted<br>(data bug) | 0.2383     | 13.2%  | 8.2%    | 7.5900      | significant | 0.020        |

PSI per feature:

| Profile      | LIMIT_BAL | AGE          | PAY_0 | utilisation_ratio | payment_ratio | max_delay    |
|--------------|-----------|--------------|-------|-------------------|---------------|--------------|
| normal       | 0.015     | 0.030        | 0.006 | 0.029             | 0.013         | 0.004        |
| drifted 0.05 | 0.051     | 0.037        | 0.006 | 0.067             | <b>0.189</b>  | 0.004        |
| drifted 0.15 | 0.057     | 0.045        | 0.006 | 0.261             | <b>1.152</b>  | 0.004        |
| drifted 1.0  | 2.043     | 1.708        | 0.826 | 3.092             | <b>4.251</b>  | 0.783        |
| unfair       | 0.015     | 0.030        | 0.279 | 0.029             | 0.288         | <b>0.353</b> |
| defaulted    | 0.015     | <b>7.590</b> | 0.006 | 0.029             | 0.013         | 0.004        |

Selection rate (share sent to REVIEW or DECLINE) by group: normal SEX=1 0.234, SEX=2 0.216; drifted 1.0 0.869 / 0.831; unfair **0.938 / 0.216**.

Dashboard screenshots: docs/screenshots/model-behaviour-{normal,drift-mild,drifted,unfair,defaulted}.png.

#### How much of “normal” is noise

A drift score of 0.03 on training-like traffic is not zero, and it should not be. We resampled held-out test rows (same population, never used to build the reference) 300 times per window size and computed the drift score:

| Window | Mean drift score | 95th percentile | Windows above 0.10 |
|--------|------------------|-----------------|--------------------|
| 100    | 0.156            | 0.260           | 88%                |
| 200    | 0.077            | 0.115           | 12%                |
| 400    | 0.039            | 0.060           | 0%                 |
| 2000   | 0.010            | 0.015           | 0%                 |

Our normal run (0.030 at 400) sits inside that noise. The 200-row minimum is roughly where the number starts to mean something, and even at 200 the maximum over six features crosses the moderate

threshold one window in eight. We kept `min_size=200` as specified, but on a low-volume service we would require 400 rows before `ModerateFeatureDrift` may page.

## 4. What the results show

---

### Which signal moved first

PSI. At strength 0.05 the drift score is 0.19, six times the normal run and inside the moderate band. Over the same run the score mean moved by 0.008 and the decision mix by half a point (DECLINE 8.0% to 8.5%, REVIEW 14.2% to 14.0%). The standard error of an 8% share over 400 requests is 1.4 points, so the decision mix is indistinguishable from normal. At strength 0.15 the drift score is already 1.15, deep in the significant band, while DECLINE has moved 1.8 points, about one standard error and nowhere near the 20% alert line. The decision mix only becomes unambiguous at full strength (54.8%).

That gap is the argument for input monitoring: the shift that sends PSI into “act” is a shift the business metric cannot yet see.

### PSI sensitivity is not uniform

`payment_ratio` leads every drifted run (0.19, 1.15, 4.25). It is the ratio of two quantities the shift moves in opposite directions (payments down, bills up), so it compounds both changes, and its quantile bins are narrow where most applicants sit. `utilisation_ratio` (bills up, limit down) is second. Raw `AGE` and `LIMIT_BAL` move least per unit of strength.

`PAY_0` and `max_delay` stay at exactly their normal values (0.006 and 0.004) at strengths 0.05 and 0.15. That is not PSI being blind: the generator adds `round(bump × strength)` months of arrears, which rounds to zero up to strength 0.25, so those inputs did not change at all. At full strength they move least (0.83 and 0.78) because they have only four bins and the top bin (two or more months late) absorbs the shift.

One global threshold across all features is therefore blunt. A ratio feature reaches the moderate band from a shift that barely registers on the raw columns; a coarse discrete feature can only move in large steps. The per-feature panel matters more than the single drift score.

### Drift and fairness are different signals

The unfair run reads 0.35 on drift, “significant”, the same band as the 0.15 and full drift runs. The aggregate cannot tell them apart. Two panels can:

- **PSI per feature.** In the unfair run `LIMIT_BAL`, `AGE` and `utilisation_ratio` are exactly at their normal values; only the payment-history features moved (`max_delay` 0.35, `payment_ratio` 0.29, `PAY_0` 0.28). The population did not change; one aspect of some applicants did.
- **Selection rate by group.** `SEX=2` is flagged at 0.216, identical to the normal run. `SEX=1` is flagged at 0.938. The gap is 0.722 against a 0.019 baseline. In the full drift run both groups rose together (0.87 and 0.83) and the gap stayed at 0.038.

The drift score says *something moved*; the per-feature PSI says *what*; the selection rate by group says *who absorbed it*. Only the last is a fairness signal.

## Which signal told us what was wrong

For the drifted runs, the per-feature PSI: a broad shift across limit, age and both ratios looks like a different population (younger, more stretched applicants), the case that might justify retraining. For the unfair run, the selection rate by group. For the defaulted run, the per-feature PSI again, and it pointed away from retraining (section 5).

## 5. Alert design

| Rule                    | Expression (short)                                              | for | Severity | signal   |
|-------------------------|-----------------------------------------------------------------|-----|----------|----------|
| ModerateFeatureDrift    | <code>ml_drift_score &gt; 0.10</code>                           | 15m | warning  | leading  |
| SignificantFeatureDrift | <code>ml_drift_score &gt; 0.25</code>                           | 15m | critical | leading  |
| DriftWindowTooSmall     | <code>ml_drift_window_size &lt; 200</code>                      | 30m | info     |          |
| DecisionMixShift        | DECLINE rate / all decisions rate (30m) > 0.20                  | 30m | warning  | output   |
| FairnessGapWidened      | <code>ml_fairness_gap &gt; 0.10</code>                          | 20m | critical | fairness |
| PredictionErrorsRising  | <code>sum(rate(ml_prediction_errors_total[5m])) &gt; 0.1</code> | 5m  | critical | service  |
| ExplanationLatencyHigh  | <code>p95 ml_explain_duration_seconds &gt; 1s</code>            | 10m | warning  | service  |

The four disciplines:

- **for: longer than one scrape.** The PSI is recomputed over a rolling window, so a short burst of unusual traffic moves it for a few scrapes and washes out. 15 minutes for drift, 20 for fairness, 30 for the decision mix. The noise table shows why this matters most for small windows.
- **A ratio, not a raw count.** `DecisionMixShift` divides DECLINE decisions by all decisions, so it means the same at 10 rps and 1000 rps. Our `ml_alert_tests.yml` checks both sides: 1000 decisions/min at an 8% DECLINE share never fires; 10 decisions/min at 30% fires after 30 minutes. `PredictionErrorsRising` is the one deliberate absolute rate. The ratio version already exists as `HighErrorRate` over HTTP 5xx, and a ratio against `ml_predictions_total` would go silent exactly when every call fails, because that labelled counter has no series until the first success. The 0.1/s floor also stops one stray error on a quiet service from paging.
- **Severity.** Critical when a person is needed now: significant drift, a fairness gap (it has legal weight), model calls failing. Warning for “investigate in working hours”. Info for “not measuring”, which is not a problem but must not be read as “fine”.
- **A description saying what to do.** Each names the next query to run (`ml_feature_drift_psi` by feature, `ml_selection_rate` by group, `ml_prediction_errors_total` by error\_type) and the decision it informs.

All twelve rules (five given, seven ours) pass `promtool test rules on alert_tests.yml` and our `ml_alert_tests.yml`, which adds one firing and one non-firing case for each of the three ML rules the

course tests do not cover.

## Which signal would have paged us

Measured values against these thresholds:

| Profile      | Would page                                                                                            | Would not                     |
|--------------|-------------------------------------------------------------------------------------------------------|-------------------------------|
| normal       | nothing                                                                                               |                               |
| drifted 0.05 | ModerateFeatureDrift (warning)                                                                        | decision mix, fairness        |
| drifted 0.15 | SignificantFeatureDrift (critical)                                                                    | decision mix (9.8%), fairness |
| drifted 1.0  | SignificantFeatureDrift;<br>DecisionMixShift if sustained 30 min                                      | fairness (0.038)              |
| unfair       | SignificantFeatureDrift and<br><b>FairnessGapWidened</b> (critical);<br>DecisionMixShift if sustained |                               |
| defaulted    | SignificantFeatureDrift (critical)                                                                    | everything else               |

Two timing details. The drift and fairness gauges describe the last N requests, not the last N minutes, so after a 15-second burst they keep their value and the 15- and 20-minute for: windows elapse with no further traffic. `DecisionMixShift` is built on `rate()` and needs the shift to last 30 minutes of real traffic, so a short burst never triggers it. That is intended for a lagging business signal, and it means a short load run demonstrates the drift and fairness alerts but not this one.

## Before retraining: is the drift a data bug?

The `defaulted` profile sends normal applicants except that every `age` is 35, as an upstream form with a default value would. The result is the textbook data-bug signature: AGE PSI 7.59, every other feature at exactly its normal value, score mean 0.238 and DECLINE 8.2% (unchanged). On the dashboard: one red bar and five green ones.

A real population shift moves correlated features together: in the full drift run younger applicants also had lower limits and higher utilisation, and all six features moved. One feature far above the rest, with its correlated features flat and the decision mix unchanged, is almost never a population shift.

`SignificantFeatureDrift` pages either way, and its description tells the on-call person to check exactly this before triggering a retrain. Retraining on the defaulted data would teach the model that `age` carries no information; the fix belongs in the pipeline that fills the field.

## 6. Limitations

- **PSI is univariate.** If income and age kept their distributions but their relationship inverted, every PSI would stay near zero. A classifier trained to tell reference rows from live rows would catch that, at more cost.
- **Labels arrive late, and never for declines.** The selection-rate gap is demographic parity: it says one group is flagged more often, not whether the flagged applicants were riskier. Equal-

opportunity style checks need default labels, which come a month later and only for approved applicants. Any retraining set is biased towards past approvals; a model that declines a group stops receiving the evidence that would correct it.

- **The window counts requests, not time.** On a quiet service the last 400 requests can be days old and the PSI keeps describing them.
- **The window lives in one process.** With several replicas each would report its own drift on a slice of traffic; production would compute drift from a shared store or from histograms aggregated in Prometheus.
- **Dashboard rates over short bursts are approximate.** `rate()` extrapolates each series separately, so on a 15-second run the dashboard's DECLINE share can read a few points off the exact count (37.9% vs 32.0% in the unfair screenshot, where the end of the previous run is also inside the 2-minute window). `/monitoring` and the load generator's counts are the exact numbers.